

Tuples

A tuple is a Python data type used to create an ordered sequence of values. Tuples can contain a mixed set of data types like this:

```
developer = ('Alice', 34, 'Rust Developer')
```

Tuples are similar to lists, but while lists are a mutable data type, tuples are immutable. This means that the elements in a tuple cannot be changed once it's created.

If you try to update one of the items in the tuple, you will get a `TypeError`:

```
programming_languages = ('Python', 'Java', 'C++', 'Rust')
programming_languages[0] = 'JavaScript'
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
```

```
"""
```

To access an element from a tuple, you can use bracket notation and the index number:

```
developer = ('Alice', 34, 'Rust Developer')
developer[1] # 34
```

If you need to access elements starting from the end of a tuple, then you can use negative indexing. Here is an example of using a negative index to access the second to last element in a tuple:

```
numbers = (1, 2, 3, 4, 5)
numbers[-2] # 4
```

If you try to pass in an index number that exceeds or equals the length of the tuple, then you will receive an `IndexError` like this:

```
numbers = (1, 2, 3, 4, 5)
numbers[7]

"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
IndexError: list index out of range
"""
```

Another way to create a tuple is by using the `tuple()` constructor like this:

```
developer = 'Jessica'
tuple(developer) # ('J', 'e', 's', 's', 'i', 'c', 'a')
```

For the `tuple()` constructor, you can pass in different iterables like strings, lists and even other tuples.

To check if an item is in a tuple, you can use the `in` keyword like this:

```
programming_languages = ('Python', 'Java', 'C++', 'Rust')

'Rust' in programming_languages # True
'JavaScript' in programming_languages # False
```

You can also unpack items from a tuple just like you did with lists:

```
developer = ('Alice', 34, 'Rust Developer')
name, age, job = developer

print(name) # 'Alice'
print(age) # 34
print(job) # 'Rust Developer'
```

In this example, `name` has the value `'Alice'`, `age` has the value `34`, and `job` has the value `'Rust Developer'`.

If you need to collect any remaining elements from a tuple, you can use the asterisk (*) operator like this:

```
developer = ('Alice', 34, 'Rust Developer')
name, *rest = developer
```

```
print(name) # 'Alice'
print(rest) # [34, 'Rust Developer']
```

Here, `name` has the value `'Alice'`, and `rest` is a list comprised of the number `34` and the string `'Rust Developer'`.

Just like with a list, you can use the slice operator on a tuple to extract a portion of it. Here is an example of extracting the items `'pie'` and `'cookies'` into a separate tuple:

```
desserts = ('cake', 'pie', 'cookies', 'ice cream')
desserts[1:3] # ('pie', 'cookies')
```

Remember that the first number represents the starting index for the extraction while the second number represents the ending index. But note that the item at the ending index is not included in the extracted tuple.

If you need to remove an item from a tuple, that isn't possible because tuples are immutable. So this example, will produce an error:

```
developer = ('Jane Doe', 23, 'Python Developer')
del developer[1]
```

```
"""
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
```

```
TypeError: "tuple" object doesn't support item deletion
```

```
"""
```

So when might you use a tuple over a list?

If you need a dynamic collection of elements where you can add, remove and update elements, then you should use a list. If you know that you are working with a fixed and immutable collection of data, then you should use a tuple.

Common Methods for Tuples

In the previous lesson, you learned how to work with the tuple data type. In this lesson, you'll learn about some common methods you'll use when working with tuples.

The first method we will cover is `count()`. This method is used to determine how many times an item appears in a tuple. Here is an example of checking how many times the string `Rust` appears in a tuple named `programming_languages`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')  
programming_languages.count('Rust') # 2
```

Since `Rust` appears twice in the tuple, the `count()` method returns the number `2`. If the specified item in the `count()` function is not present at all in the tuple, then the return value is `0`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')  
programming_languages.count('JavaScript') # 0
```

If no arguments are passed into the `count()` function, then Python raises a `TypeError`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust')  
programming_languages.count()
```

```
"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
TypeError: tuple.count() takes exactly one argument (0 given)
"""
```

The next method we will look at is the `index()` method. This method is used to find the index where a particular item is present in a tuple. Here is an example of using the `index()` method to find the index for the string `Java`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',
                          'Rust')
programming_languages.index('Java') # 1
```

If the specified item cannot be found, then Python raises a `ValueError`:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',
                          'Rust')
programming_languages.index('JavaScript')
```

```
"""
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
ValueError: tuple.index(x): x not in tuple
"""
```

Another thing you can do with the `index()` method is to pass in optional start and stop index arguments. Here is an example of passing in an optional start index:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',
                          'Rust', 'Python')
programming_languages.index('Python', 3) # 5
```

In this example, we are specifying where to start searching for the string `Python`. By passing in the number `3` as the second argument to the `index()` function, we are specifying to start searching at index `3`. Since `Python` appears twice in the tuple, the `index()` function will return index `5` instead of index `2` because of the use of the optional start index argument.

You can also pass in an optional stop index. Here is a modified example of specifying a start and stop index:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust', 'Python', 'JavaScript', 'Python')  
programming_languages.index('Python', 2, 5) # 2
```

Now the result is index `2` because we are starting the search at index `2`, and searching up to, but not including, index `5`.

Another commonly used function used with tuples is the `sorted()` function. In a previous lesson you learned about the `sort()` method for lists. Well, the `sorted()` function can be used on any iterable including tuples.

Here is an example of creating a new list of numbers using the `sorted()` function:

```
numbers = (13, 2, 78, 3, 45, 67, 18, 7)  
sorted(numbers) # [2, 3, 7, 13, 18, 45, 67, 78]
```

The `sorted()` function will always create a new list of the sorted values. This differs from the `sort()` method which sorts the elements of a list in place and does not return a new list.

If you need to customize the sorting behavior for an iterable, you can use the optional `reverse` and `key` arguments. Here is an example of using `key` argument to sort items in a tuple by length:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust', 'Python')  
sorted(programming_languages, key=len)
```

```
# Result
```

```
# ['C++', 'Rust', 'Java', 'Rust', 'Python', 'Python']
```

If you want to create a new list of values in reverse order, then you can use the `reverse` argument like this:

```
programming_languages = ('Rust', 'Java', 'Python', 'C++',  
                          'Rust', 'Python')
```

```
print(sorted(programming_languages, reverse=True))
```

```
# Result
```

```
# ['Rust', 'Rust', 'Python', 'Python', 'Java', 'C++']
```

Tuples are a common data type in Python. Understanding how to work with them, along with some helpful methods and functions, will help you write more efficient code.

Dictionaries

In Python, dictionaries are built-in data structures that store collections of key-value pairs. They work very similarly to real dictionaries, where you search for a word to find its corresponding meaning.

With Python dictionaries, you use a key to find its corresponding value. You should use dictionaries when you need to associate values to unique keys. This is helpful when you need to find a value fast based on the key, and when you need to represent structured data.

This is the general syntax of a Python dictionary:

Example Code

```
dictionary = {  
    key1: value1,  
    key2: value2  
}
```

First, we find the variable that holds the dictionary. You don't necessarily need to assign the dictionary to a variable, but it's very common to do this to keep it in memory and use it later in the code.

Then that's followed by curly braces, which are sometimes called curly brackets. And within the curly braces, there are key-value pairs.

Each key is associated with a value, so you can use the key to access that value. After each value, except the last one, there's a comma to separate the different key-value pairs. Keys must be unique in the dictionary, and they must be an immutable data type. However, the values can be repeated, and they can be of any data type.

Here we have an example of a dictionary that stores information about a Margherita pizza recipe:

Example Code

```
pizza = {  
    'name': 'Margherita Pizza',
```



```
'price': 8.9,  
'calories_per_slice': 250,  
'toppings': ['mozzarella', 'basil']  
}
```

The dictionary is assigned to the `pizza` variable. It has four key-value pairs: `name`, `price`, `calories_per_slice`, and `toppings`.

Another alternative would be using the `dict()` constructor, which builds the dictionary from a sequence of key-value pairs.

This would be the equivalent syntax for our pizza example. We pass a list of tuples as argument to the `dict()` constructor. These tuples contain the key as the first element and the value as the second element.

Example Code

```
pizza = dict([('name', 'Margherita Pizza'), ('price', 8.9),  
( 'calories_per_slice', 250), ('toppings', ['mozzarella',  
'basil'])])
```

To access the value of a key-value pair, you can use this syntax, known as bracket notation. It's the name of the variable that holds the dictionary, followed by square brackets, and the key you want to access within the square brackets:

Example Code

```
dictionary[key]
```

In our pizza example, if you want to access the value of `name`, you would write the name of the variable, `pizza`, followed by square brackets, and the key, `name`, within quotes:

Example Code

```
pizza['name']
```

This will evaluate to:

Example Code

```
'Margherita Pizza'
```

To update a value, you just need to add the assignment operator, followed by the new value.

If the key doesn't exist in the dictionary, a new key-value pair will be created. In recent versions of Python, dictionaries preserve the order of insertion. This is helpful when you need to iterate over the dictionary:

Example Code

```
pizza['name'] = 'Margherita'
```

Now the value of the key `name` is `'Margherita'`:

Example Code

```
print(pizza['name']) # 'Margherita'
```

Dictionaries also have helpful methods to perform common operations.

The `.get()` method retrieves the value associated with a key. It's similar to the bracket notation that we just used, but its advantage is that you can set a default value, so you won't get an error if the key doesn't exist:

Example Code

```
dictionary.get(key, default)
```

In this example, if the `toppings` key doesn't exist, it will return an empty list, which is the default value that we are passing here as the second argument. But if `toppings` does exist, it will return that value:

Example Code

```
pizza.get('toppings', []) # ['mozzarella', 'basil']
```

The `.keys()` and `.values()` methods return a view object with all the keys and values in the dictionary, respectively:

Example Code

```
pizza.keys()
```

```
# dict_keys(['name', 'price', 'calories_per_slice'])
```

```
pizza.values()
```

```
# dict_values(['Margherita Pizza', 8.9, 250])
```

A view object is just a way to see the content of a dictionary without creating a separate copy of the data.

The `.items()` method returns a view object with all the key-value pairs in the dictionary, including both the keys and the values:

Example Code

```
pizza.items()
```

```
# dict_items([('name', 'Margherita Pizza'), ('price', 8.9), ('calories_per_slice', 250)])
```

The `.clear()` method removes all the key-value pairs from the dictionary:

Example Code

```
pizza.clear()
```

The `.pop()` method removes the key-value pair with the key that you specify as the first argument and returns its value. If the key doesn't exist, it returns the default value that you specify as the second argument. If the key doesn't exist and you don't pass a default value, a `KeyError` is raised:

Example Code

```
pizza.pop('price', 10)
```

```
pizza.pop('total_price') # KeyError
```

In Python 3.7 and more recent versions, the `.popitem()` method removes the last inserted item:

Example Code

```
pizza.popitem()
```

And finally, the `.update()` method updates the key-value pairs with the key-value pairs of another dictionary. If they have keys in common, their values are overwritten.

In this example, we are updating the `pizza` dictionary. The `price` key exists in both of them, so its value will be replaced with `15`.

But `total_time` is new, so it will be added to the `pizza` dictionary as a new key-value pair:

Example Code

```
pizza.update({ 'price': 15, 'total_time': 25 })
```

This is the new dictionary with the updated `price` and the new `total_time`. Notice how the price is now `15` and `total_time` is a new key-value pair:

Example Code

```
{  
    'name': 'Margherita Pizza',  
    'price': 15,  
    'calories_per_slice': 250,  
    'toppings': ['mozzarella', 'basil'],  
    'total_time': 25  
}
```

These are some of the most commonly used dictionary methods, but there are many more. Choosing the right one can be helpful for performing complex operations efficiently.

Common Techniques to Loop Over a Dictionary

You can loop over a dictionary if you need to access and process its key-value pairs. This is helpful for updating their values or applying some logic to them.

Let's take a look at some of the techniques you can use.

Let's say that we have a `products` dictionary that associates every product with its price:

Example Code

```
products = {  
    'Laptop': 990,  
    'Smartphone': 600,  
    'Tablet': 250,  
    'Headphones': 70,  
}
```

If we want to offer a 20% discount on all our products, we can loop over all the key-value pairs and modify the prices.

The `.values()`, `.keys()`, and `.items()` methods are essential for these techniques. We covered them briefly in a previous lesson.

They return a view object with the values, keys, and key-value pairs of the dictionary. You can use these view objects in `for` loops to iterate over the elements.

For example, you can iterate over all the values of the dictionary like this.

You write `for`, the loop variable (`price` in this case), `products.values()` to get all the values of the `products` dictionary, a colon, and then the body of the loop, where you can apply any logic to the values. In this case, we are printing them.

The loop variable will take each one of the values, one per iteration:

Example Code

```
for price in products.values():  
    print(price)
```

And here is the output. As you can see, each value is printed to the console, one by one:

Example Code

```
990  
600  
250  
70
```

This works exactly the same for `.keys()` if you need to iterate over the keys of a dictionary. You just need to iterate over `products.keys()` or `products` directly, and assign a descriptive name for the loop variable:

Example Code

```
for product in products.keys():  
    print(product)  
  
# Or  
  
for product in products:  
    print(product)
```

This is the output. Each key is printed to the console, one at a time:

Example Code

```
Laptop  
Smartphone  
Tablet  
Headphones
```

And this works exactly the same for key-value pairs if you need to iterate over the keys and their corresponding values simultaneously. You just need to iterate over `products.items()`:

Example Code

```
for product in products.items():  
    print(product)
```

This is the output. Now you get individual tuples with the keys and their corresponding values:

Example Code

```
('Laptop', 990)  
( 'Smartphone', 600)  
( 'Tablet', 250)  
( 'Headphones', 70)
```

If you want to store the key and the value in separate loop variables, you just need to define them and separate them with a comma. Then, you can use them in the body of the loop.

Here, we are defining a `product` loop variable and a `price` loop variable. Each one will hold its corresponding value. It's important to define them in order – the key first, and then the value:

Example Code

```
for product, price in products.items():  
    print(product, price)
```

This is the output. We are printing them side by side, but you can use these values as you need them in your code.

Example Code

```
Laptop 990  
Smartphone 600  
Tablet 250
```

Headphones 70

Now that you know more about this, we can go back to our initial example. If we want to offer a 20% discount, we would multiply each price by `0.8` and reassign it as the value of that product key.

We could also round the result down if we want to work with integers:

Example Code

```
products = {  
    'Laptop': 990,  
    'Smartphone': 600,  
    'Tablet': 250,  
    'Headphones': 70,  
}  
  
for product, price in products.items():  
    products[product] = round(price * 0.8)  
  
print(products)
```

Then, if we print the dictionary, we would get these key-value pairs with the discounted prices:

Example Code

```
{  
    'Laptop': 792,  
    'Smartphone': 480,  
    'Tablet': 200,  
    'Headphones': 56  
}
```


And finally, if you need to iterate over the key-value pairs while keeping track of a counter, you can call the `enumerate()` function. This counter essentially acts as a sort of "index" or "count" for that element within the loop.

The function returns an `enumerate` object, which assigns an integer to each key-value pair, like a counter. You can start the counter from any number, but by default, it starts from 0.

Here, we are iterating over the keys of the `products` dictionary:

Example Code

```
for product in enumerate(products):  
    print(product)
```

But the `enumerate()` function also assigns an integer to each key, so we get tuples with the integer and the key.

Here is the output:

Example Code

```
(0, 'Laptop')  
(1, 'Smartphone')  
(2, 'Tablet')  
(3, 'Headphones')
```

If you need to, you can assign these values to separate loop variables. Here, we have two loop variables (`index` and `product`). This is what you will commonly see and use when you work with `enumerate()`:

Example Code

```
for index, product in enumerate(products):  
    print(index, product)
```

If you need to iterate over the values, you can replace `products` by `products.values()`:

Example Code

```
for price in enumerate(products.values()):  
    print(price)
```

The output will have the index and the price in each tuple:

Example Code

```
(0, 990)  
(1, 600)  
(2, 250)  
(3, 70)
```

You can assign them to separate loop variables as well:

Example Code

```
for index, price in enumerate(products.values()):  
    print(index, price)
```

This will be the output. You can use them as you need to in your code:

Example Code

```
0 990  
1 600  
2 250  
3 70
```

And with `products.items()`, you can get the entire key-value pair in addition to the "index" or "counter":

Example Code

```
for index, product in enumerate(products.items()):  
    print(index, product)
```

In this example, we get the index followed by a tuple that contains the key and the value of the corresponding key-value pair:

Example Code

```
0 ('Laptop', 990)
1 ('Smartphone', 600)
2 ('Tablet', 250)
3 ('Headphones', 70)
```

To customize the initial value of the count, you can pass a second argument to `enumerate()`. For example, here we are starting the count from 1:

Example Code

```
for index, product in enumerate(products.items(), 1):
    print(index, product)
```

You can see this change in the output. Now the first integer is 1 instead of 0:

Example Code

```
1 ('Laptop', 990)
2 ('Smartphone', 600)
3 ('Tablet', 250)
4 ('Headphones', 70)
```

This works with any variation we've seen so far. You just need to pass the initial number as the second argument.

There are many techniques to loop over a dictionary. These are some common ways, and you'll need to choose the best one for your project.

Sets

Sets are one of Python's built-in data structures. One of the core characteristics of sets is that they don't store duplicate values. If you try to add a duplicate value to a set, only one of them will be stored.

Sets are mutable and unordered, which means that their elements are not stored in any specific order, so you cannot use indices or keys to access them. They can only contain values of immutable data types like numbers, strings, and tuples. And they support mathematical set operations, including union, intersection, difference, and symmetric difference.

To define a set, you just need to write its elements within curly braces and separate them with commas. This is an example of a set of numbers:

Example Code

```
my_set = {1, 2, 3, 4, 5}
```

One quirk of working with sets is that, if you ever need to define an empty set, you must use the `set()` function. If you just write empty curly braces, like `{}`, Python will automatically create a dictionary.

Example Code

```
set() # Set  
{ }  # Dictionary
```

You can add an element to a set with the `.add()` method, and pass in the new element as argument:

Example Code

```
my_set.add(6)
```

In our example, the new set would be:

Example Code

```
{1, 2, 3, 4, 5, 6}
```

If you try to add an element that is already in the set, only one will be kept. In this case, we already have the number 5 in the set:

Example Code

```
my_set.add(5)
```

So the set will not change:

Example Code

```
{1, 2, 3, 4, 5, 6}
```

To remove an element from the set, you have two options. You can either use the `.remove()` method or the `.discard()` method, and pass in the element that you want to remove as argument.

The `.remove()` method will raise a `KeyError` if the element is not found, while the `.discard()` method will not:

Example Code

```
my_set.remove(4)
my_set.discard(4)
```

The `.clear()` method removes all the elements from the set:

Example Code

```
my_set.clear()
```

Python sets also have powerful methods that perform common mathematical set operations.

The `.issubset()` and the `.issuperset()` methods check if a set is a subset or superset of another set, respectively.

Here, we are checking if `your_set` is a subset of `my_set`, which is `False` because not all the elements of `your_set` are in `my_set`.

We are also checking if `my_set` is a superset of `your_set`. This is also `False` because `my_set` does not have all the elements of `your_set`:

Example Code

```
my_set = {1, 2, 3, 4, 5}
your_set = {2, 3, 4, 6}

print(your_set.issubset(my_set)) # False
print(my_set.issuperset(your_set)) # False
```

The `.isdisjoint()` method checks if two sets are disjoint, which means they don't have any elements in common. In this case, that's `False` because `my_set` and `your_set` do have common elements – 2, 3, and 4:

Example Code

```
print(my_set.isdisjoint(your_set)) # False
```

The union operator `|` returns a new set with all the elements from both sets:

Example Code

```
my_set | your_set # {1, 2, 3, 4, 5, 6}
```

The intersection operator `&` returns a new set with only the elements that the sets have in common:

Example Code

```
my_set & your_set # {2, 3, 4}
```

The difference operator `-` returns a new set with the elements of the first set that are not in the other sets. In this example, the numbers 1 and 5 are in `my_set` but NOT in `your_set`:

Example Code

```
my_set - your_set # {1, 5}
```

The symmetric difference operator `^` returns a new set with the elements that are either in the first or the second set, but not both. In this case, 1 and 5 are

in `my_set` but not in `your_set`, so they are included. And the number 6 is in `your_set` but not in `my_set`, so it's included as well:

Example Code

```
my_set ^ your_set # {1, 5, 6}
```

Each one of these operators also has its corresponding compound assignment operator if you add the equal sign next to it. These operators automatically assign the resulting set to the first set in the expression:

Example Code

```
|=  &=  -=  ^=
```

For example, the `-=` operator finds the difference between the sets and updates the first set with that result:

Example Code

```
my_set -= your_set
```

After this, `my_set` will be updated to `{1, 5}`:

Example Code

```
print(my_set) # {1, 5}
```

You can check if an element is in a set or not with the `in` operator. Here, we are checking if 5 is in `my_set`. The result will be a boolean value `True` or `False`:

Example Code

```
print(5 in my_set)
```

And those are the fundamentals of sets. They are very helpful when you don't need to store the values in any specific order, and when you only need to store unique values.